Laboratório Prolog · Aula 13

Luiz Otavio Roder · RA 24159236-2 · Turma ESOFT-C
Gerado em 19/10/2026, 19:34:00 · tempo registrado no cronômetro: 0:00

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) encontra L = scheme. paradigma(java, logico) falha (false), porque o fato registrado para Java é paradigma(java, orientado_a_objetos).

Saída do ola.pl

```
SAÍDA ESPERADA — deduzida dos fatos do ola.pl (não executada no OneCompiler nesta sessão):
?- linguagem(L).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog, P).
   paradigma(prolog, logico)
?- paradigma(L, funcional).
   paradigma(scheme, funcional)
?- paradigma(java, logico).
   false.

Com os dois experimentos descomentados:
?- linguagem(X1).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(java, X2).
   paradigma(java, orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual é o paradigma associado a Prolog: o Prolog procura um valor para a variável P e encontra P = logico. paradigma(prolog, logico) já apresenta os dois argumentos fixos e pergunta apenas se a afirmação pode ser comprovada; nesse caso, a resposta é verdadeira.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

A resposta false significa que o programa não consegue provar paradigma(java, logico) com os fatos e as regras cadastrados. Pela hipótese do mundo fechado, o que não é demonstrável é tratado como falso na base, mas isso não equivale a uma prova universal de que Java jamais possa empregar lógica. A base afirma somente paradigma(java, orientado_a_objetos).

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas: lucia, marta, tereza, joana, bia, ana e dora. A consulta (mae(M,F), mulher(F)) terá 4 pares: (lucia, tereza), (marta, joana), (tereza, bia) e (joana, ana).

Saída do familia.pl

```
SAÍDA ESPERADA — inferida de familia.pl, não capturada do OneCompiler:
--- 1. sem variavel
mae(joana,jake) → verdadeiro
mae(jake,joana) → false.
pai(maria,jake) → false.
--- 2. com variavel
mae(joana,F) → F=jake; F=ana
pai(P,jake) → P=jorge
mulher(Quem) → lucia; marta; tereza; joana; bia; ana; dora
--- 3. conjuncao
mae(joana,X), homem(X) → X=jake
pai(vern,Y), mulher(Y) → Y=joana
mae(M,Filha), mulher(Filha) → (lucia,tereza); (marta,joana); (tereza,bia); (joana,ana)

Experimentos sugeridos:
mae(joana,F1) → F1=jake; F1=ana
pai(P2,jake) → P2=jorge
mae(M3,F3), homem(F3) → (lucia,vern); (marta,paulo); (joana,jake); (dora,leo)
homem(X4), mulher(X4) → false.
mae(joana,jake), mae(joana,ana) → verdadeiro.
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta pergunta: “Quais mulheres M são mães de filhas F que também são mulheres?”. Ela encontra 4 soluções: Mae = lucia, Filha = tereza; Mae = marta, Filha = joana; Mae = tereza, Filha = bia; e Mae = joana, Filha = ana. A mesma variável Filha é compartilhada pelas duas condições.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

É a hipótese do mundo fechado: tudo que não pode ser demonstrado usando os fatos e regras existentes é tratado como falso. Assim, pai(maria, jake) falha porque não há fato correspondente; o pai cadastrado é jorge. Em situações reais, uma base pode estar incompleta, então “não encontrado” não significa necessariamente “não existe”, o que pode produzir decisões erradas.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Na ordem em que os fatos aparecem no código: lucia, marta, tereza, joana, bia, ana e dora. Isso ocorre porque o Prolog procura as cláusulas de cima para baixo e apresenta uma solução por vez, retornando para encontrar a próxima por backtracking.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) retorna X = paulo, porque paulo é irmão de joana, mãe de jake, e homem(paulo) é fato. primo(leo, X) encontra X = jake e X = ana, pois paulo (pai de leo) e joana (mãe de jake e ana) são irmãos.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falham f(X,X) = f(a,b), porque X precisaria ser a e b simultaneamente; [A|B] = [], porque uma lista não vazia não unifica com a vazia; e 2+3 = 5, porque = não realiza cálculo. f(X,X) = f(a,Y) unifica com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
RESULTADOS ESPERADOS DE regras.pl E unificacao.pl — análise do código; não são logs
capturados do OneCompiler.
regras.pl:
avo(A, jake) → A=marta; A=vern
avo(osvaldo, N) → N=joana; N=paulo; N=bia
irmao(jake, I) → I=ana
tio(T, jake) → T=paulo
primo(leo, P) → P=jake; P=ana
ancestral(osvaldo, D) → D=vern; tereza; joana; paulo; jake; ana; leo; bia
ancestral(A, jake) → A=joana; jorge; lucia; marta; osvaldo; vern (a ordem é de busca, não de
parentesco)
ancestral(jake, _) → false.
Sem X \== Y na regra irmao/2: irmao(jake, jake) passa a ser aceito, além de irmao(jake, ana).

unificacao.pl:
a = a → unifica
a = b → NAO unifica
X = a → unifica (X=a)
f(X,b) = f(a,Y) → unifica (X=a, Y=b)
f(X,X) = f(a,b) → NAO unifica
f(X,X) = f(a,Y) → unifica (X=a, Y=a)
pai(X,jake) = pai(jorge,Y) → unifica (X=jorge, Y=jake)
[H|T] = [1,2,3] → unifica (H=1, T=[2,3])
[H|T] = [] → NAO unifica
[X,Y|R] = [a,b,c,d] → unifica (X=a, Y=b, R=[c,d])
2+3 = 5 → NAO unifica (sem avaliar a expressão).
A apresentação literal das variáveis depende do interpretador.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque ancestral/2 precisa reconhecer tanto o parentesco direto quanto o indireto, em qualquer número de gerações. O caso base é ancestral(X,Y) :- progenitor(X,Y): X é pai ou mãe de Y. O caso recursivo é ancestral(X,Y) :- progenitor(X,Z), ancestral(Z,Y): X é progenitor de Z e Z é ancestral de Y. A recursão permite percorrer gerações sem escrever uma regra para cada distância.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, a regra permite que a mesma pessoa seja considerada irmã de si própria, pois os dois conjuntos de fatos da mãe e do pai unificam com a mesma pessoa. A consulta irmao(jake, _) passa a incluir jake além de ana. X \== Y exige que os dois termos já instanciados sejam diferentes e impede esse resultado incorreto.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X,X) = f(a,b), o primeiro argumento obriga X=a, mas o segundo exigiria X=b; como a e b são átomos diferentes, há conflito e a unificação falha. Em f(X,X) = f(a,Y), o primeiro argumento liga X=a e o segundo liga Y ao mesmo valor. Portanto, X=a e Y=a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

A relação avo(X,Z) não fixa quem é entrada e quem é saída: com Z=jake ela encontra os avós; com X=osvaldo ela encontra os netos, usando a mesma regra e a unificação. Uma função Java ou Scheme comum recebe argumentos e calcula um retorno em uma direção; seria necessário escrever procedimentos extras para a busca inversa. O Prolog descreve uma relação e explora as possíveis ligações de variáveis.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

São 4 respostas, porque os três elementos podem ser divididos entre prefixo X e sufixo Y em quatro posições: X=[], Y=[1,2,3]; X=[1], Y=[2,3]; X=[1,2], Y=[3]; X=[1,2,3], Y=[].

Saída do listas.pl

```
SAÍDA ESPERADA — obtida por análise de listas.pl (não executada nesta sessão):
--- membro
membro(b,[a,b,c]) → verdadeiro
membro(z,[a,b,c]) → false.
membro(X,[a,b,c]) → X=a; X=b; X=c
--- tamanho
tamanho([a,b,c],N) → N=3
tamanho([],N) → N=0
--- concatena
concatena([1,2],[3,4],L) → L=[1,2,3,4]
concatena(X,[3],[1,2,3]) → X=[1,2]
concatena([1],Y,[1,2,3]) → Y=[2,3]
concatena(X,Y,[1,2,3]) →
  X=[], Y=[1,2,3]
  X=[1], Y=[2,3]
  X=[1,2], Y=[3]
  X=[1,2,3], Y=[]
--- inverte
inverte([1,2,3],R) → R=[3,2,1]

Experimentos:
concatena(X1,[b,c],[a,b,c]) → X1=[a]
inverte([a,b,c,d],R2) → R2=[d,c,b,a]
tamanho([x,y],N3) → N3=2
membro(X4,[]) → false.
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([],0): lista vazia tem tamanho zero. No passo recursivo, tamanho([_|Cauda],N) chama tamanho(Cauda,M) e depois calcula N is M+1. Em tamanho([a,b,c],N), ficam pendentes três somas de 1: primeiro o tamanho de [b,c], depois [c] e finalmente []. Ao atingir 0, o programa retorna 1, depois 2, depois 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque concatena/3 descreve todas as divisões possíveis da lista em um prefixo e um sufixo: há um corte antes do primeiro item, dois entre itens e um após o último, totalizando 4. O anexar de Scheme da Aula 12 normalmente recebe duas listas conhecidas e devolve a lista concatenada; ele não busca automaticamente todos os pares de listas que produziriam um resultado determinado.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking. Primeiro, membro(X,[X|_]) unifica X com o primeiro elemento. Ao solicitar outra solução, o Prolog retorna ao ponto de escolha e tenta a cláusula recursiva membro(X,[_|Cauda]), repetindo com a cauda. Assim ele gera a, b e c sem um laço explícito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2+3 liga X ao termo composto +(2,3), sem efetuar a soma; X is 2+3 calcula e liga X ao número 5. fatorial(N,120), com N não instanciado, provoca erro de instanciação ao tentar avaliar N > 0: essa definição exige que N seja

informado.

Saída do aritmetica.pl

```
SAÍDA ESPERADA — inferida de aritmetica.pl (não coletada no OneCompiler):
--- = unifica, is calcula
X = 2+3 → X fica ligado à estrutura 2+3
X is 2+3 → X=5
5 =:= 2+3 → verdadeiro
2+3 = 5 → false.
X is 7 mod 3 → X=1
X is 7 // 2 → X=3
X is 2 ^ 10 → X=1024
--- fatorial, fib e soma
fatorial(5,F) → F=120
fatorial(0,F) → F=1
fib(10,F) → F=55
soma([1,2,3,4],S) → S=10
--- aritmetica nao e relacional
fatorial(N,120) → ERRO: instantiation_error
X is Y+1, com Y livre → ERRO: instantiation_error

Experimentos extras (valores previstos):
X1 is 5*4+2 → 22
X2 = 5*4+2 → termo 5*4+2 (sem cálculo)
fatorial(10,F3) → 3628800
fib(20,F4) → 6765.
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

O operador = faz unificação, não soma: X = 2+3 liga X ao termo 2+3. Já is avalia a expressão numérica à direita: X is 2+3 produz X=5. O operador =:= compara os valores depois de avaliá-los: 5 =:= 2+3 é verdadeiro, enquanto 4 =:= 2+3 é falso. Em contraste, 2+3 = 5 falha porque um termo composto e um número não são estruturalmente iguais.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

concatena(X,[3],[1,2,3]) é uma relação estrutural: o Prolog pode unificar listas e descobrir X sem precisar calculá-lo antecipadamente. Já fatorial(N,120) chega ao teste N > 0 com N livre e o operador aritmético exige um número conhecido, causando instantiation_error. Aritmética comum com is, > etc. não funciona automaticamente nos dois sentidos.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

As duas definições usam recursão com caso base fatorial(0)=1 e passo que multiplica N pelo fatorial de N-1. Em Haskell, fact n é uma função que recebe n e devolve um resultado. Em Prolog, fatorial(N,F) é um predicado com o resultado expresso por outro argumento e com is para calcular; apesar da notação relacional, a definição ainda exige N conhecido por causa da aritmética.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) falha (false), pois tenta negar casado(X) com X ainda livre. solteiro1(X) encontra osvaldo, paulo e jake. maximo2(5,3,M) produz duas respostas indevidas: M=5 e M=3. maximo3(5,3,3) também dá verdadeiro (erro lógico), pois a primeira cláusula não unifica e o corte não é alcançado.

Saída de controle.pl e mensagem de ancestral_ruim

```
RESULTADOS ESPERADOS — análise de controle.pl e laco_infinito.pl; não são logs coletados de
execução.
controle.pl:
--- A) backtracking
Tentativas amigo(X), amigo(Y) incluem ana/ana (X\==Y falha), ana/bia (bebidas diferentes),
ana/caio (cafe/cafe, resposta), as combinações de bia (sem outra bebida igual) e caio/ana
(cafe/cafe, resposta). Resultado: (ana,caio) e (caio,ana).
--- B) negacao
solteiro1(X) → X=osvaldo; X=paulo; X=jake
solteiro2(X) → false.
--- C) corte
maximo1(5,3,M) → M=5
maximo2(5,3,M) → M=5; M=3
maximo3(5,3,M) → M=5
maximo1(5,3,3) → false.
maximo3(5,3,3) → verdadeiro (erro da regra)

laco_infinito.pl:
ancestral_bom(osvaldo,jake)? → sim
ancestral_ruim(osvaldo,jake)? → recursão à esquerda infinita; erro esperado: Fatal Error:
local stack overflow (mensagem exata depende do interpretador).
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores, prevejo 24 colorações válidas para as oito regiões e as fronteiras informadas. Com 2 cores, nenhuma (0), pois PR, SP e MS formam um triângulo de regiões vizinhas e requerem três cores diferentes.

Saída do mapa.pl

```
SAÍDA ESPERADA — contagem conferida por enumeração independente das restrições do mapa; saída
do Prolog não capturada:
--- coloracoes com 3 cores [RS,SC,PR,SP,MS,MG,RJ,ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas colorações com 3 cores?
24
--- e com 2 cores?
false.
--- e com 4 cores?
1728

As duas primeiras listas seguem a ordem das escolhas especificada no programa.
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog começa por X=ana e Y=ana. A meta X \== Y falha, pois uma pessoa não deve formar par consigo mesma. Ele volta à escolha de amigo(Y), tenta Y=bia, mas não encontra uma bebida B comum (cafe ≠ cha). Ao tentar Y=caio, encontra cafe para ambos e produz (ana,caio). Depois volta a amigo(X), chegando a X=caio e Y=ana para encontrar (caio,ana).

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em solteiro1(X), homem(X) primeiro escolhe pessoas concretas e a negação verifica para cada uma se casado(X) não pode ser provado; daí saem osvaldo, paulo e jake. Já solteiro2(X) faz \+ casado(X) com X sem valor; como existe

ao menos um casado (vern), a negação falha globalmente. Osvaldo sai como solteiro porque não existe casado(osvaldo) na base, embora pai(osvaldo,vern) seja fato. Ser pai não prova ser casado.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte está dentro de maximo3(X,Y,X) :- X >= Y, !. Na consulta maximo3(5,3,3), a cabeça da primeira cláusula não unifica, porque precisaria que o terceiro argumento fosse 5. Assim, a execução nem chega ao corte. A segunda cláusula maximo3(_,Y,Y) aceita 3 incorretamente. Correção: deixe o resultado M livre na cabeça e faça a ligação após o corte:

maximo_corrigido(X, Y, M) :- X >= Y, !, M = X.
maximo_corrigido(_, Y, M) :- M = Y.
Agora maximo_corrigido(5,3,3) falha, e maximo_corrigido(5,3,M) retorna apenas M=5.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim é recursiva à esquerda: sua primeira meta é ancestral_ruim(X,Z), antes de consumir um fato progenitor. A busca em profundidade chama a própria regra repetidamente sem avançar até um caso base, estourando a pilha. Em ancestral_bom, a cláusula base é tentada primeiro e a recursão vem depois de progenitor(X,Z), que avança uma geração; com essa árvore familiar finita e sem ciclos ela termina.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Encontrei, pela contagem das combinações que obedecem às 11 fronteiras: 24 colorações com 3 cores, 0 com 2 cores e 1.728 com 4 cores. O programa é declarativo porque especifica somente as relações e restrições que tornam uma coloração válida (membro e cores distintas em vizinhos). Ele não explicita os laços de busca: a execução e o backtracking são responsabilidade do Prolog.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional, uma função como anexar do Scheme recebe listas e calcula uma lista resultante. No lógico, concatena(X,Y,Z) descreve uma relação que também consegue descobrir X e Y a partir de Z. O Prolog responde por unificação e backtracking, em vez de apenas calcular um retorno.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Primeira deficiência: a ordem das metas altera o comportamento com negação; em controle.pl, solteiro2(X) falha, ao contrário de solteiro1(X). Segunda: a busca em profundidade é sensível à ordem das regras; em laco_infinito.pl, ancestral_ruim entra em recursão à esquerda e estoura a pilha mesmo existindo uma definição lógica válida.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para sistemas especialistas, dedução de parentesco, consulta a regras, análise de relações e problemas de restrições como colorir mapas, pois basta declarar fatos e condições. Preferiria Java para aplicações corporativas, interfaces, serviços com muitas operações de entrada e saída e sistemas que exigem controle explícito de estado, integração e desempenho previsível.

Exercícios opcionais

Opcionais: resultado dos testes

```
RESULTADO ESPERADO APÓS INSERIR AS SOLUÇÕES (a suíte Prolog não foi executada nesta sessão):
Exercício 1: 6 verificações → ok
Exercício 2: 3 verificações → ok
Exercício 3: 4 verificações → ok
```

```
Exercício 4: 5 verificações → ok
Exercício 5: 4 verificações → ok
Exercício 6: 3 verificações → ok
Exercício 7: 7 verificações → ok
TOTAL: 32 verificações devem exibir "ok" e nenhuma deve exibir "FALHOU".

Consulta complementar do desafio SEND+MORE=MONEY (solução informada no arquivo da aula):
  9567
+ 1085
= 10652

Observação: estes são resultados derivados dos códigos e dos testes incluídos no enunciado,
não uma transcrição de execução real do OneCompiler.
```

E1 Regras de família: código escrito

```
% Exercicio 1 - irma e tia
irma(X, Y) :- irmao(X, Y), mulher(X).
tia(X, Y) :- progenitor(P, Y), irma(X, P).
```

E2 Listas: código escrito

```
% Exercicio 2 - ultimo elemento
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
% Exercicio 3 - soma dos quadrados e pares
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, R), S is R + H*H.
pares([], []).
pares([H|T], [H|R]) :- 0 is H mod 2, pares(T, R).
pares([H|T], R) :- H mod 2 =\= 0, pares(T, R).
```

E4 Aritmética: código escrito

```
% Exercicio 4 - MDC de Euclides e potencia
mdc(A, 0, A).
mdc(A, B, D) :- B =\= 0, R is A mod B, mdc(B, R, D).
potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, P), R is B * P.
```

E5 Grafo com ciclos: código escrito

```
% Exercicio 5 - caminho dirigido com visitados
caminho(A, B) :- caminho_vis(A, B, [A]).
caminho_vis(A, B, _) :- liga(A, B).
caminho_vis(A, B, Visitados) :-
    liga(A, C),
    \+ membro(C, Visitados),
    caminho_vis(C, B, [C|Visitados]).
```

E6 Mapa menor: código escrito

```
% Exercicio 6 - mapa RS/SC/PR
mapa_sul(Paleta, [RS, SC, PR]) :-
```

```
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
% Exercicio 7 - consultas a funcionarios
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
total_depto(D, Total) :- findall(S, funcionario(_, D, S), Lista), soma(Lista, Total).
```